

TaskAPI
Sistema de Gestión de Tareas

Integrantes:

Marbin Ronaldo Meneses
Brayan Bambague Gallardo
Yordi
Jhonathan

Presentado a:

Leidy Mariani Dorado Flórez

Descripción del problema

Actualmente muchas personas gestionan sus tareas diarias de forma manual o utilizan aplicaciones que no generan recordatorios automáticos personalizados. Esto puede ocasionar que las tareas se olviden o no se realicen a tiempo.

El sistema TaskAPI busca solucionar este problema permitiendo a los usuarios registrar tareas y recibir notificaciones automáticas en Telegram cuando estas se encuentran pendientes.

De esta manera el usuario puede mantener un mejor control sobre sus actividades personales o laborales.

Arquitectura implementada

El sistema se implementa como una arquitectura distribuida basada en microservicios, en la cual cada funcionalidad principal se desarrolla como un servicio independiente.

Los servicios que componen el sistema son:

- Servicio de usuarios: encargado del registro, autenticación y gestión de usuarios.
- Servicio de tareas: encargado de la creación, consulta y actualización de tareas.
- Servicio de notificaciones: encargado de consultar tareas pendientes y enviar recordatorios a los usuarios.
- Frontend: interfaz de usuario que permite interactuar con el sistema.

Cada uno de estos servicios se ejecuta en un contenedor Docker independiente, lo que permite su aislamiento y facilita su despliegue.

La comunicación entre los servicios se realiza mediante peticiones HTTP siguiendo un estilo REST. El frontend actúa como cliente, consumiendo los servicios de usuarios y tareas, mientras que los servicios backend también se comunican entre sí para cumplir ciertas funcionalidades. Por ejemplo, el servicio de notificaciones consulta tanto el servicio de tareas como el de usuarios para enviar recordatorios.

El sistema sigue un modelo cliente-servidor, donde el frontend realiza solicitudes a los microservicios, y estos procesan la información y devuelven respuestas.

Esta arquitectura permite separar responsabilidades, facilitar el mantenimiento del sistema y representar un enfoque típico de sistemas distribuidos, donde múltiples componentes independientes colaboran para ofrecer una funcionalidad completa.

Descripción del avance

En esta etapa del proyecto se logró pasar de una propuesta conceptual a una implementación funcional del sistema TaskAPI.

Se implementaron los servicios de usuarios y tareas, permitiendo el registro, autenticación y gestión de tareas dentro del sistema. Además, se integró un mecanismo de autenticación basado en JWT para proteger los endpoints y controlar el acceso según el rol del usuario.

Como parte del avance, se desarrolló el servicio de notificaciones, el cual interactúa con los demás servicios para consultar tareas pendientes y enviar recordatorios a los usuarios.

También se incorporó una interfaz de usuario que permite consumir los servicios de forma más intuitiva.

Finalmente, se logró contenerizar cada componente utilizando Docker y se configuró Docker Compose para ejecutar todos los servicios de manera integrada, permitiendo que se comuniquen entre sí dentro de una misma red.

Funcionalidades implementadas

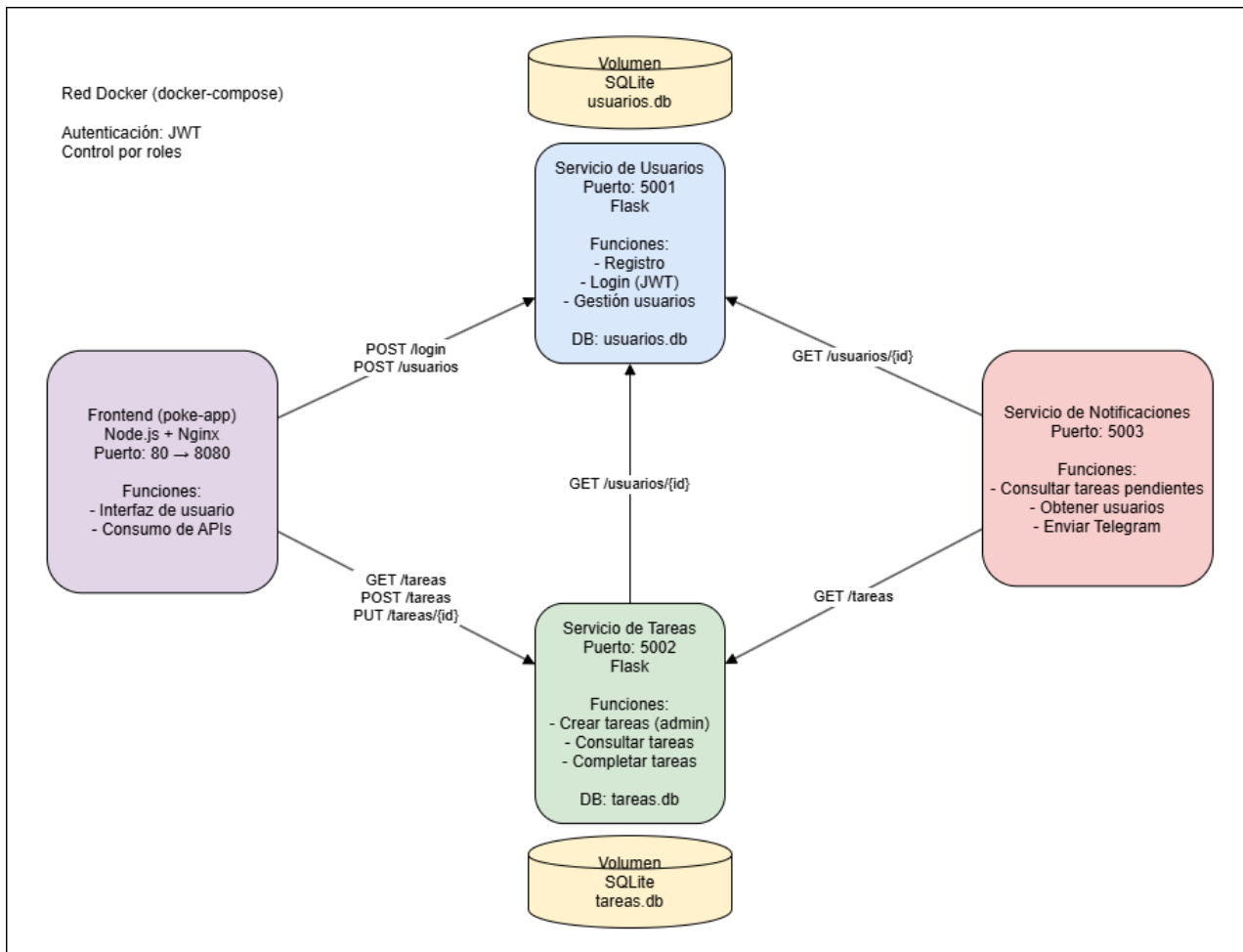
- Registro de usuarios.
- Inicio de sesión mediante autenticación con JWT.
- Creación automática de un usuario administrador al iniciar el sistema.
- Gestión de tareas:
 - Creación de tareas (solo administrador)
 - Consulta de tareas (según rol)
 - Marcado de tareas como completadas
- Asociación de tareas a usuarios.
- Envío de notificaciones de tareas pendientes mediante Telegram.
- Comunicación entre servicios mediante API REST.
- Control de acceso basado en roles (admin / usuario).
- Contenerización completa del sistema utilizando Docker.
- Persistencia de datos mediante bases de datos SQLite.

Componentes no implementados o en mejora

- Manejo avanzado de errores entre servicios.
- Implementación de una base de datos centralizada (actualmente cada servicio usa SQLite).
- Mecanismos de alta disponibilidad o escalabilidad automática.

Arquitectura actualizada

El sistema implementa una arquitectura de microservicios donde cada componente funciona de manera independiente dentro de un contenedor Docker. Se utilizó la herramienta draw.io (diagrams.net) para la construcción del diagrama actualizado del sistema.



Servicios existentes

- Servicio de Usuarios - Puerto: 5001
 - Tecnologías: Python, Flask, SQLite

Funciones:

- Registro de usuarios
 - Inicio de sesión (generación de JWT)
 - Gestión de roles (admin / usuario)
 - Creación automática de usuario administrador
-

- Servicio de Tareas - Puerto: 5002
 - Tecnologías: Python, Flask, SQLite

Funciones:

- Creación de tareas (solo admin)
 - Consulta de tareas
 - Marcado de tareas como completadas
-

- Servicio de Notificaciones - Puerto: 5003

Funciones:

- Consulta de tareas pendientes
 - Obtención de usuarios
 - Envío de notificaciones mediante Telegram
-

- Frontend (Interfaz de usuario) - Puerto: 8080
 - Tecnologías: Node.js, Nginx

Función:

- Interfaz gráfica para interacción del usuario
 - Consumo de servicios backend
-

Relación entre servicios

El sistema está compuesto por múltiples servicios que interactúan entre sí mediante solicitudes HTTP dentro de una red interna gestionada por Docker Compose.

1. Interacción del cliente (Frontend → Backend)

El frontend actúa como punto de entrada del usuario y se comunica directamente con los siguientes servicios:

- **Servicio de usuarios**
 - Registro de nuevos usuarios
 - Autenticación (login y obtención de JWT)
- **Servicio de tareas**
 - Consulta de tareas
 - Marcado de tareas como completadas

2. Comunicación entre microservicios

Los servicios backend también se comunican entre sí para cumplir ciertas funcionalidades:

- **Servicio de tareas → Servicio de usuarios**
 - Se utiliza para validar información de usuarios al asignar o consultar tareas.

3. Procesos internos (servicio de notificaciones)

El servicio de notificaciones opera de forma independiente y consulta otros servicios para ejecutar su lógica:

- **Notificaciones → Servicio de tareas**
 - Obtiene las tareas pendientes.
- **Notificaciones → Servicio de usuarios**
 - Obtiene la información de contacto de los usuarios (chat_id).
- **Notificaciones → Servicio externo (Telegram)**
 - Envía mensajes de notificación a los usuarios.

Endpoints principales

Servicio de usuarios

POST /login → autenticación y generación de token

POST /usuarios → registro de usuario

GET /usuarios → consulta de usuarios (solo admin)

Servicio de tareas

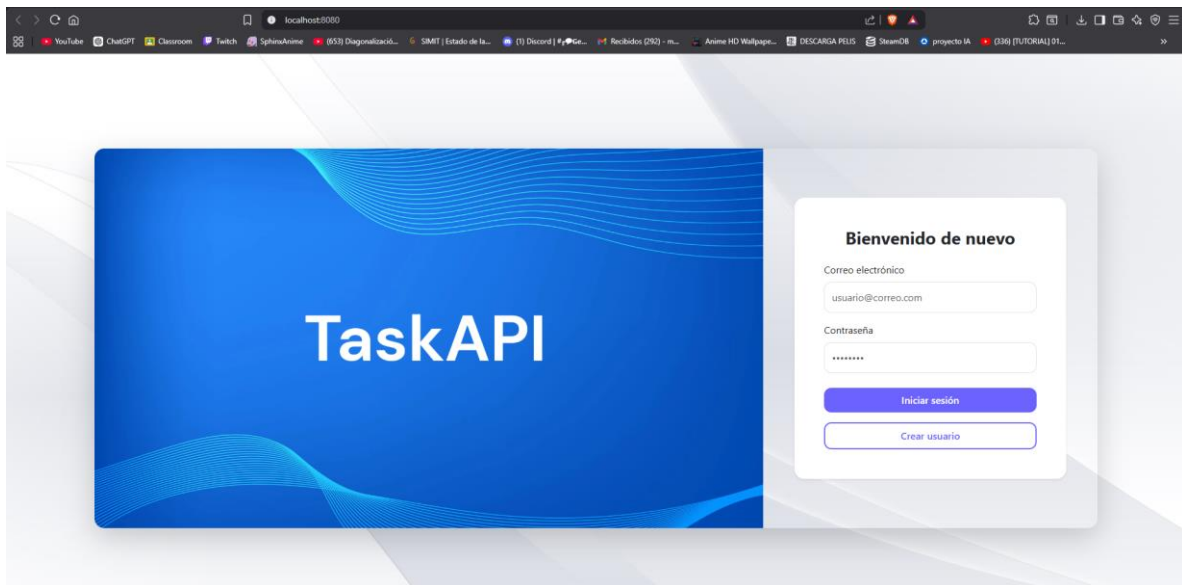
GET /tareas → consulta de tareas

POST /tareas → creación de tareas (solo admin)

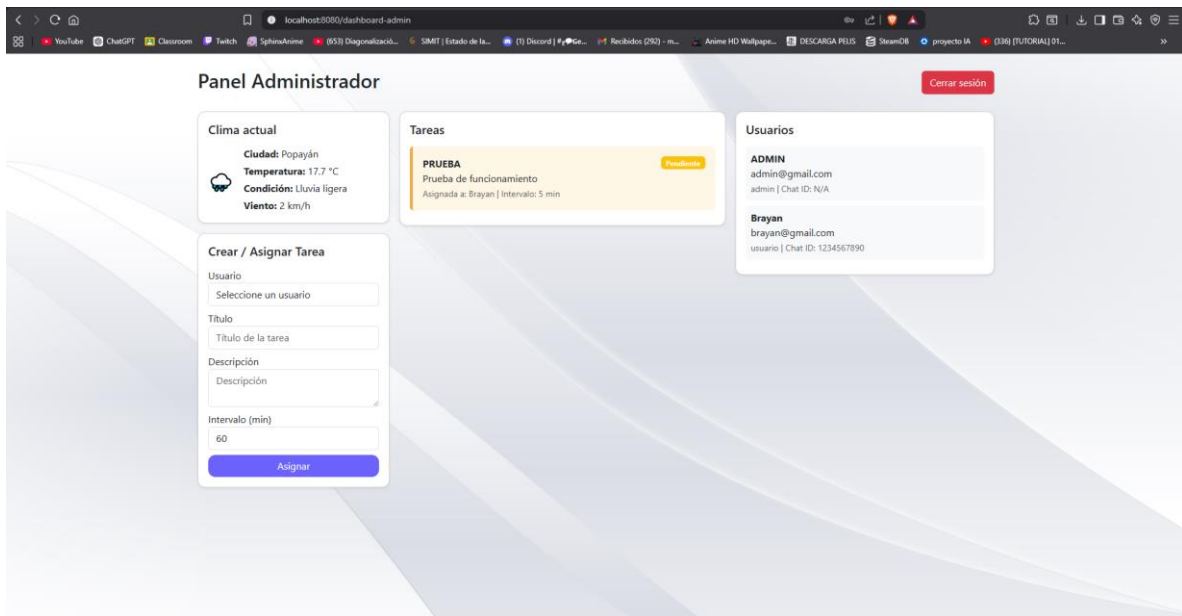
PUT /tareas/{id}/completar → marcar tarea como completada

Flujo completo de interacción (ADMIN)

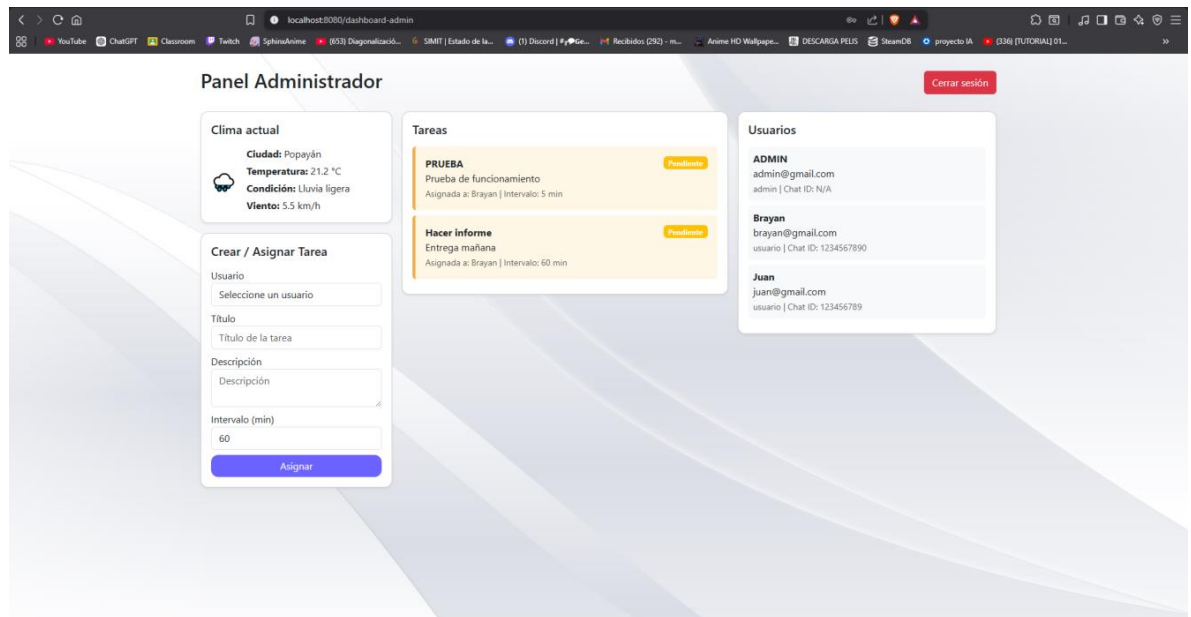
1. El admin accede al sistema mediante el frontend.



2. El admin se autentica:
 - POST /login
 - Obtiene un token JWT.
3. El frontend utiliza el token para consumir el servicio de tareas:
 - GET /tareas

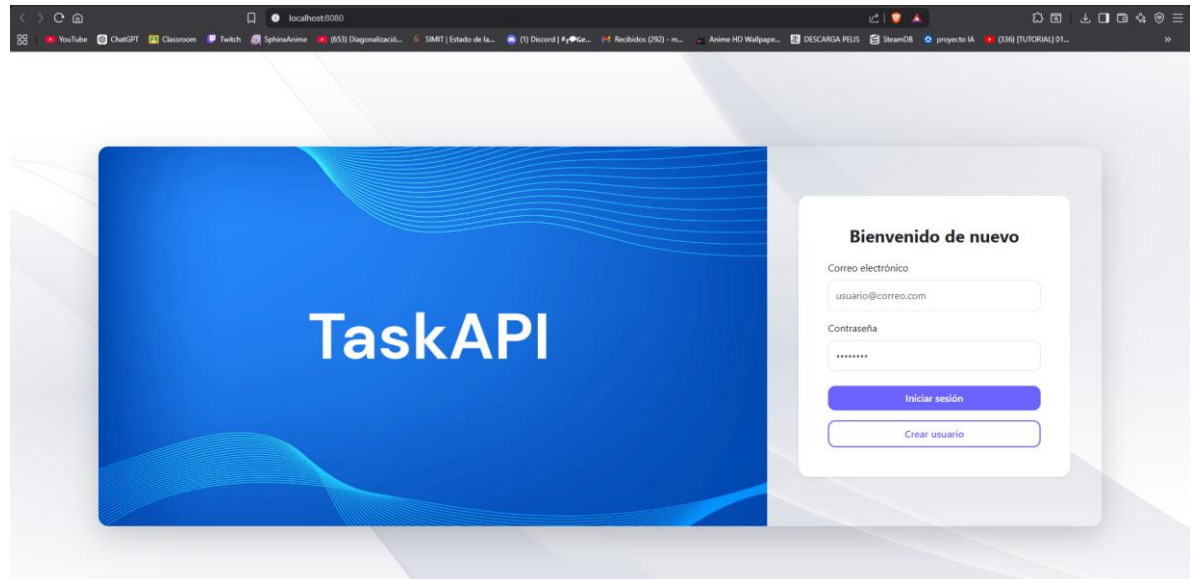


4. El administrador crea tareas:
 - a. POST /tareas



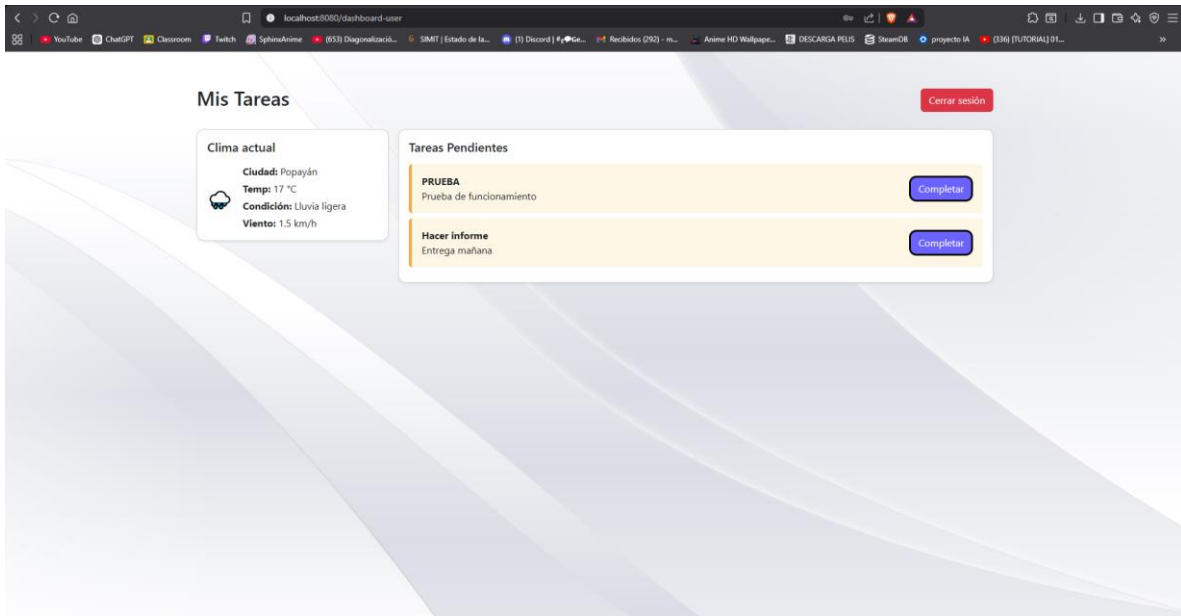
Flujo completo de interacción (USUARIO)

1. El usuario accede al sistema mediante el frontend.

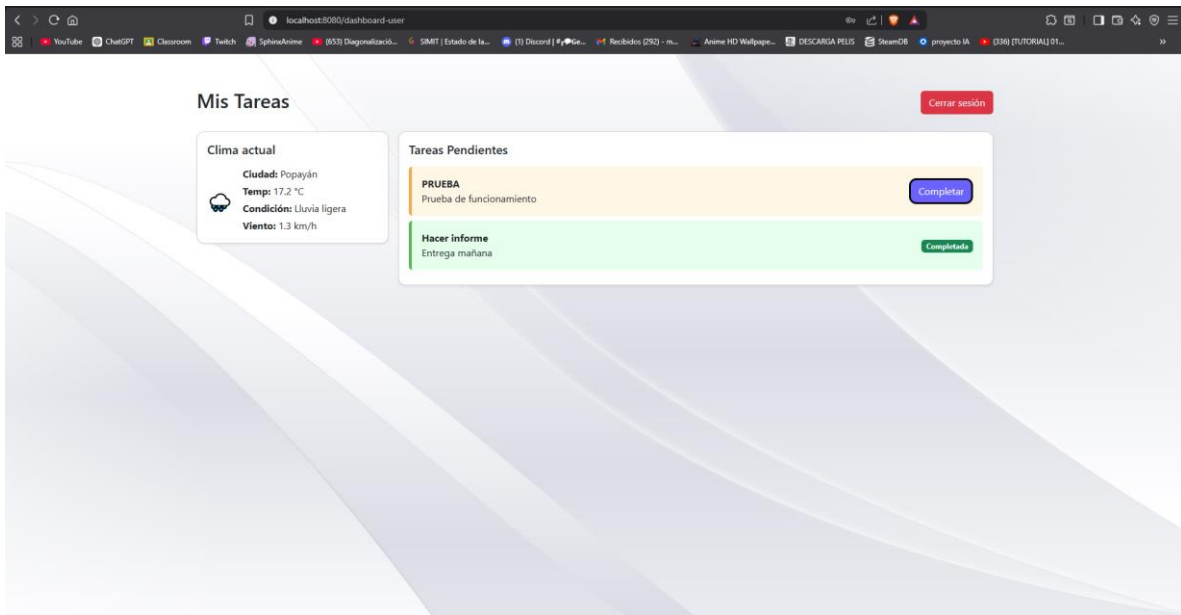


2. El usuario se autentica:
 - POST /login
 - Obtiene un token JWT.

3. El frontend utiliza el token para consumir el servicio de tareas:
- GET /tareas



4. El usuario consulta y completa tareas:
- PUT /tareas/{id}/completar

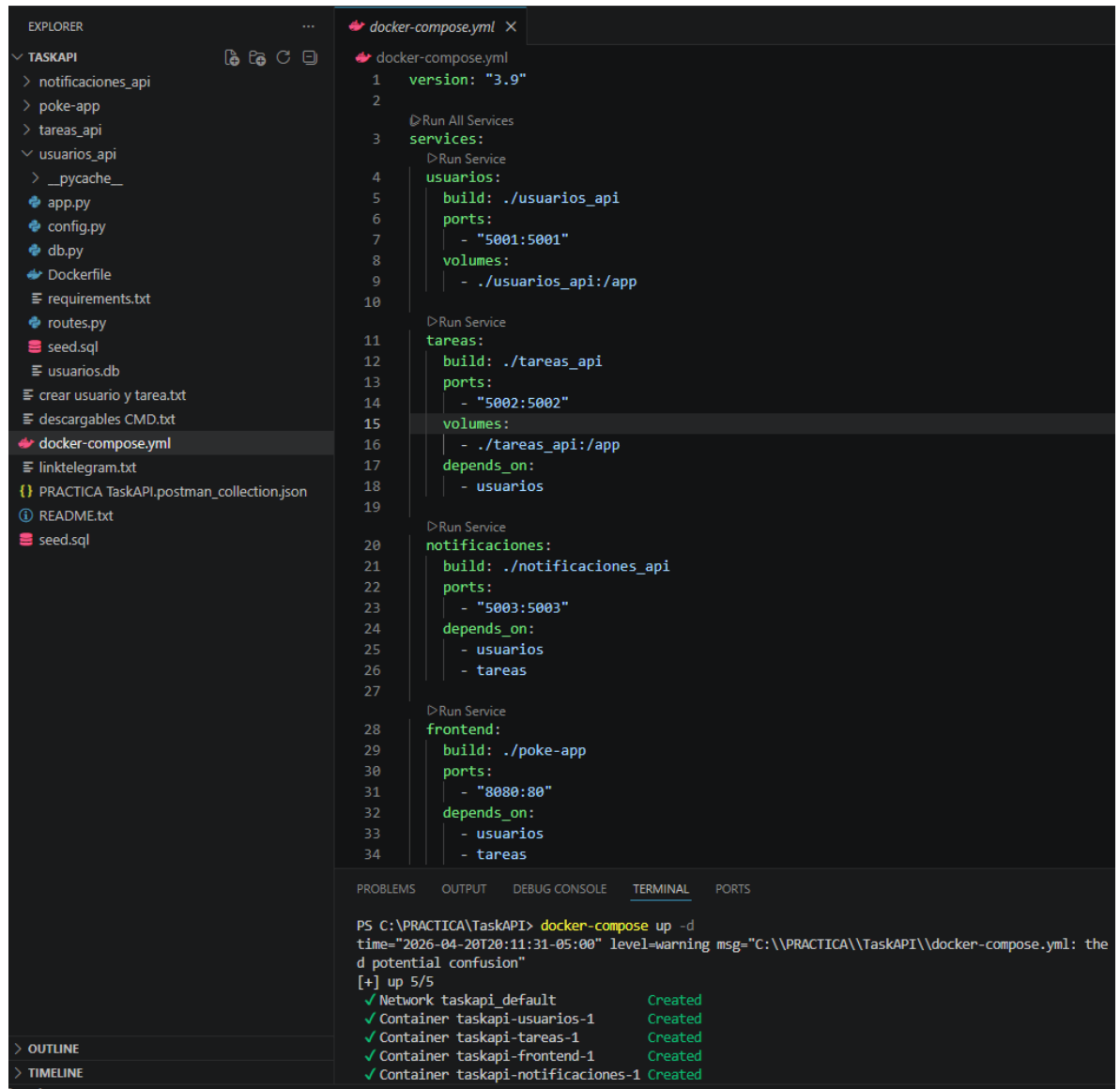


Flujo completo de interacción (NOTIFICACIONES)

El servicio de notificaciones:

- Consulta tareas pendientes (GET /tareas)
- Obtiene usuarios (GET /usuarios)
- Envía notificaciones mediante Telegram

Prototipo Técnico en ejecución

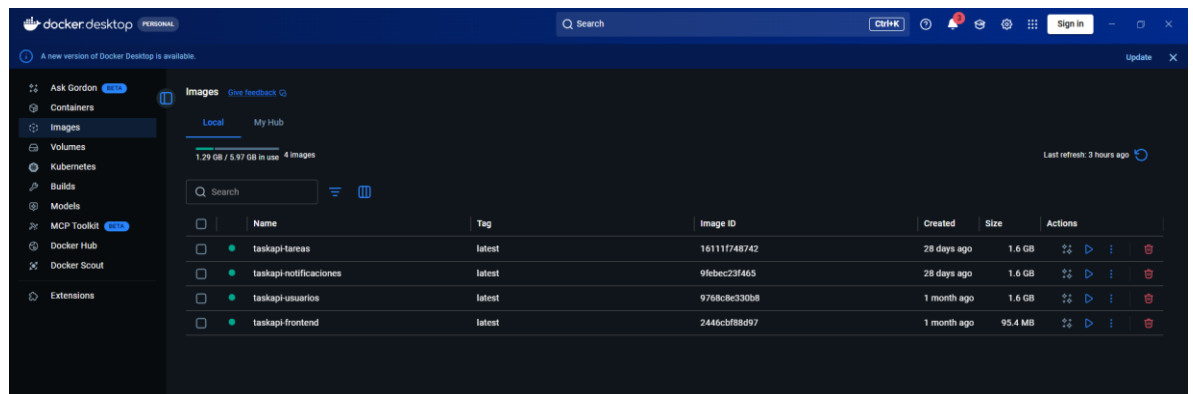


The image shows a Visual Studio Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named TASKAPI with various files including API endpoints, configuration, database, Dockerfile, requirements, routes, seed, and users. The code editor displays the docker-compose.yml file, which defines three services: usuarios, tareas, and frontend. The terminal at the bottom shows the command docker-compose up -d being executed, resulting in the creation of four containers: taskapi-default, taskapi-usuarios-1, taskapi-tareas-1, taskapi-frontend-1, and taskapi-notificaciones-1.

```
docker-compose.yml
1 version: "3.9"
2
3 services:
4   usuarios:
5     build: ./usuarios_api
6     ports:
7       - "5001:5001"
8     volumes:
9       - ./usuarios_api:/app
10
11   tareas:
12     build: ./tareas_api
13     ports:
14       - "5002:5002"
15     volumes:
16       - ./tareas_api:/app
17     depends_on:
18       - usuarios
19
20   notificaciones:
21     build: ./notificaciones_api
22     ports:
23       - "5003:5003"
24     depends_on:
25       - usuarios
26       - tareas
27
28   frontend:
29     build: ./poke-app
30     ports:
31       - "8080:80"
32     depends_on:
33       - usuarios
34       - tareas
```

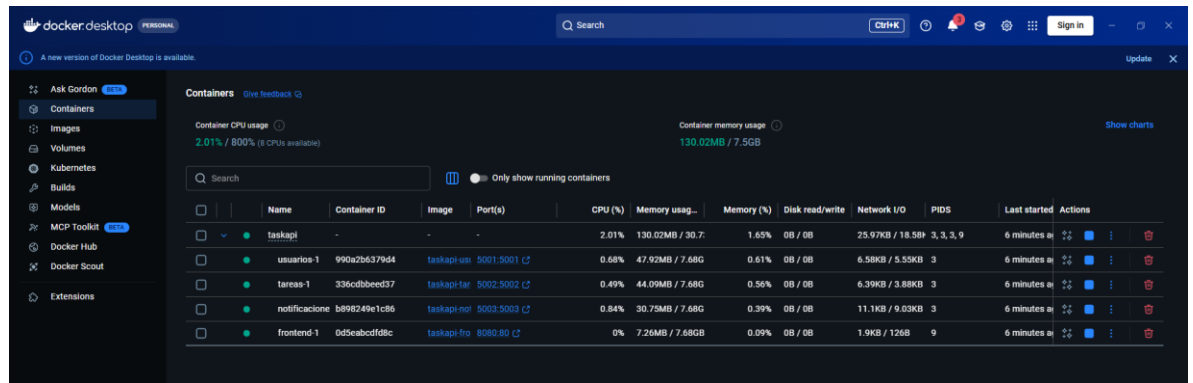
PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\PRACTICA\TaskAPI> docker-compose up -d
time="2026-04-20T20:11:31-05:00" level=warning msg="C:\\PRACTICA\\TaskAPI\\docker-compose.yml: the
d potential confusion"
[+] up 5/5
✓ Network taskapi_default Created
✓ Container taskapi-usuarios-1 Created
✓ Container taskapi-tareas-1 Created
✓ Container taskapi-frontend-1 Created
✓ Container taskapi-notificaciones-1 Created
```

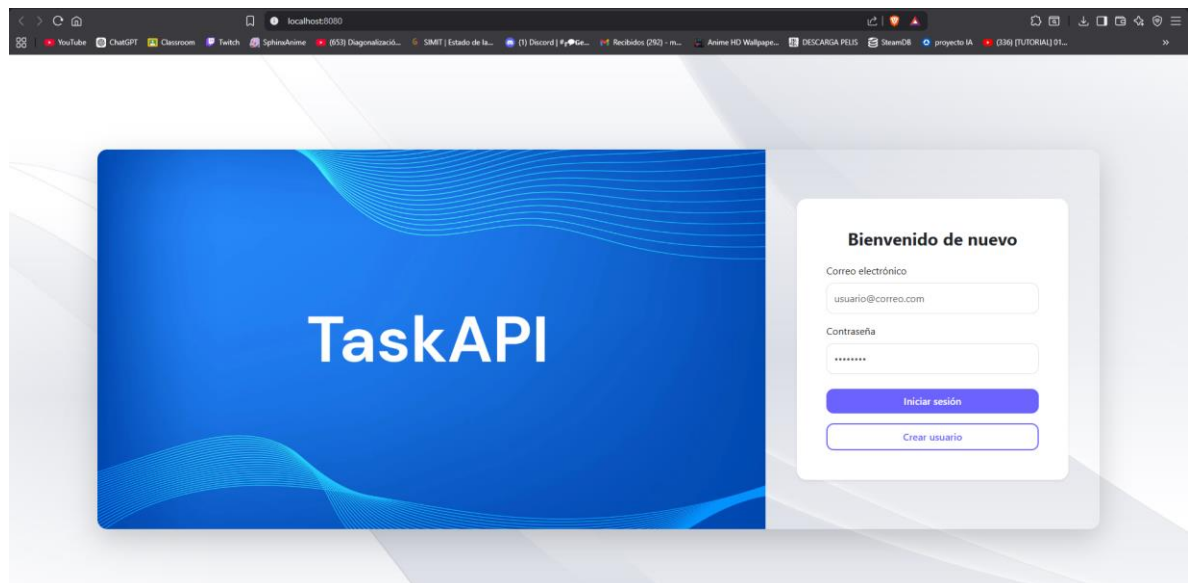
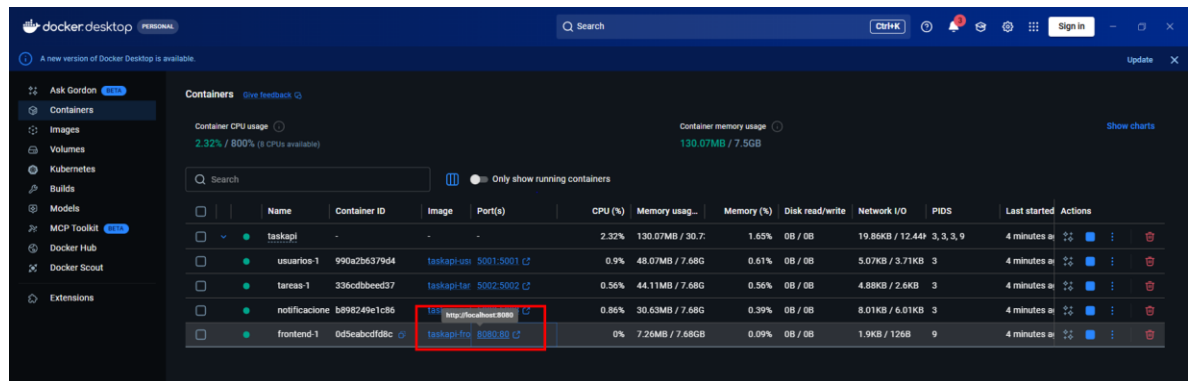


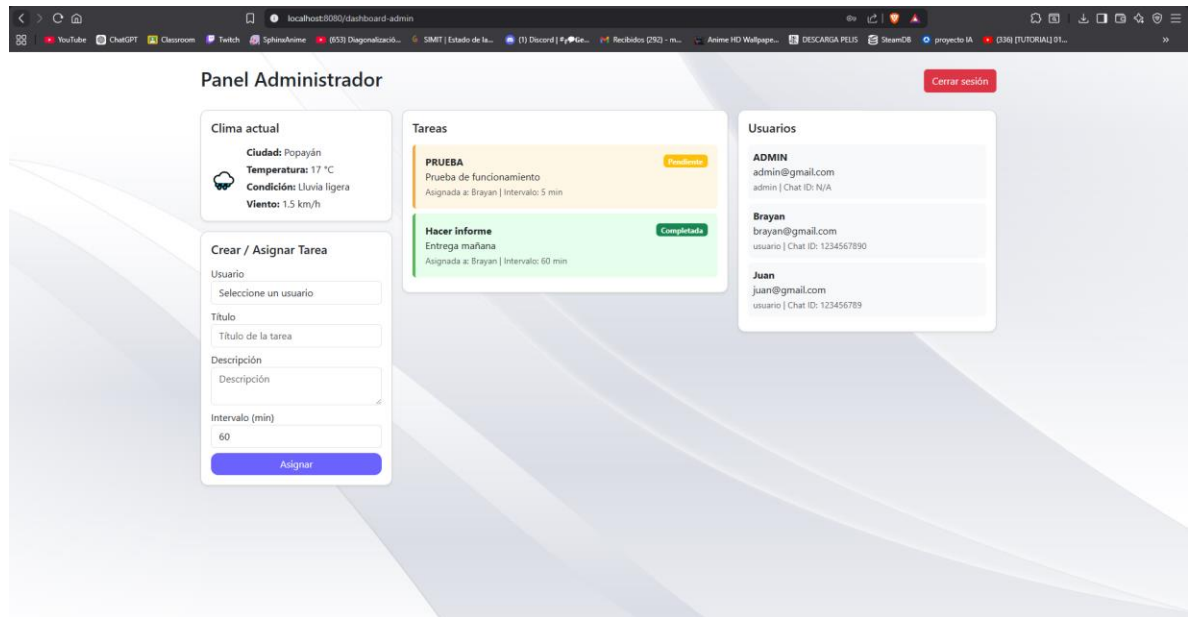
The image shows the Docker Desktop interface. The left sidebar contains navigation options: Containers, Images, Volumes, Kubernetes, Builds, Models, MCP Toolkit, Docker Hub, Docker Scout, and Extensions. The main area displays the 'Images' tab, showing a list of local images. The table includes columns for Name, Tag, Image ID, Created, Size, and Actions. The images listed are taskapi-tareas, taskapi-notificaciones, taskapi-usuarios, and taskapi-frontend.

Name	Tag	Image ID	Created	Size	Actions
taskapi-tareas	latest	16111748742	28 days ago	1.6 GB	🔍 ⏪ ⏩ 🗑️
taskapi-notificaciones	latest	9f6bec23f465	28 days ago	1.6 GB	🔍 ⏪ ⏩ 🗑️
taskapi-usuarios	latest	9768cbe330b8	1 month ago	1.6 GB	🔍 ⏪ ⏩ 🗑️
taskapi-frontend	latest	2446cbf88d97	1 month ago	95.4 MB	🔍 ⏪ ⏩ 🗑️



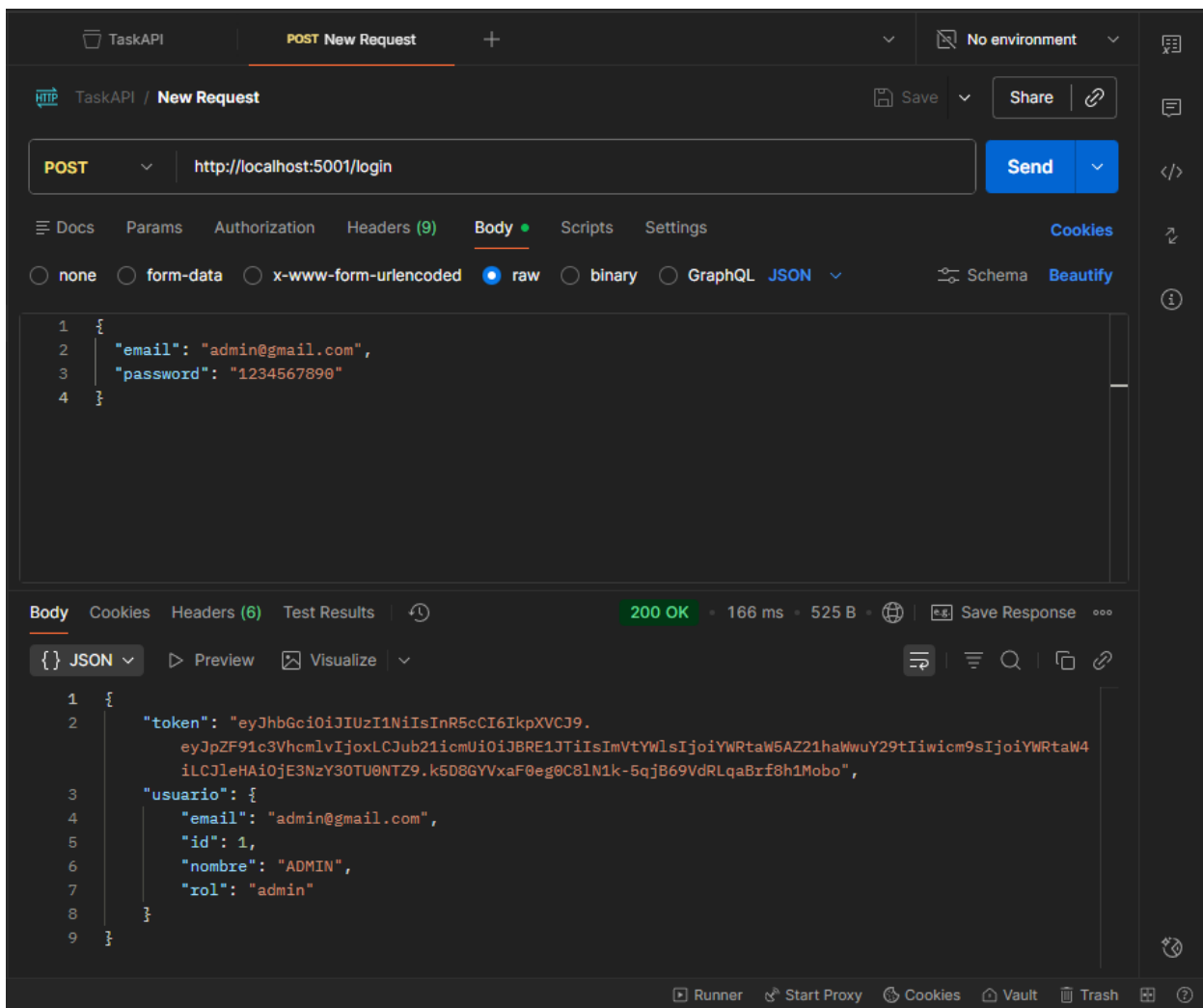
Acceso al frontend



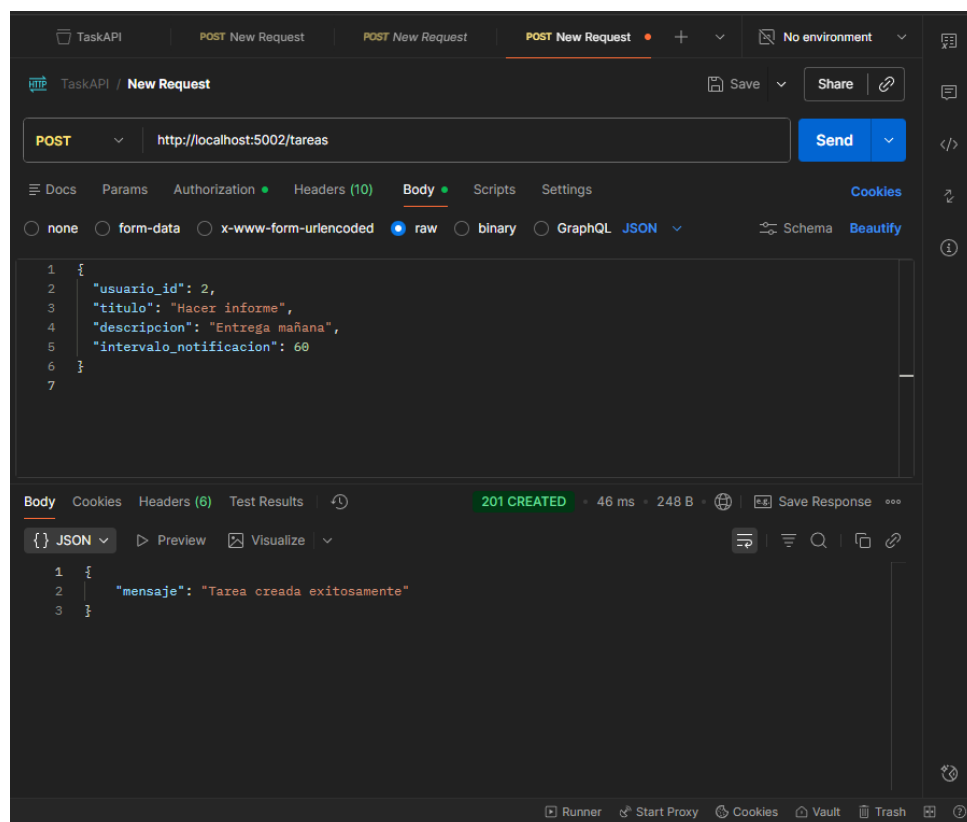
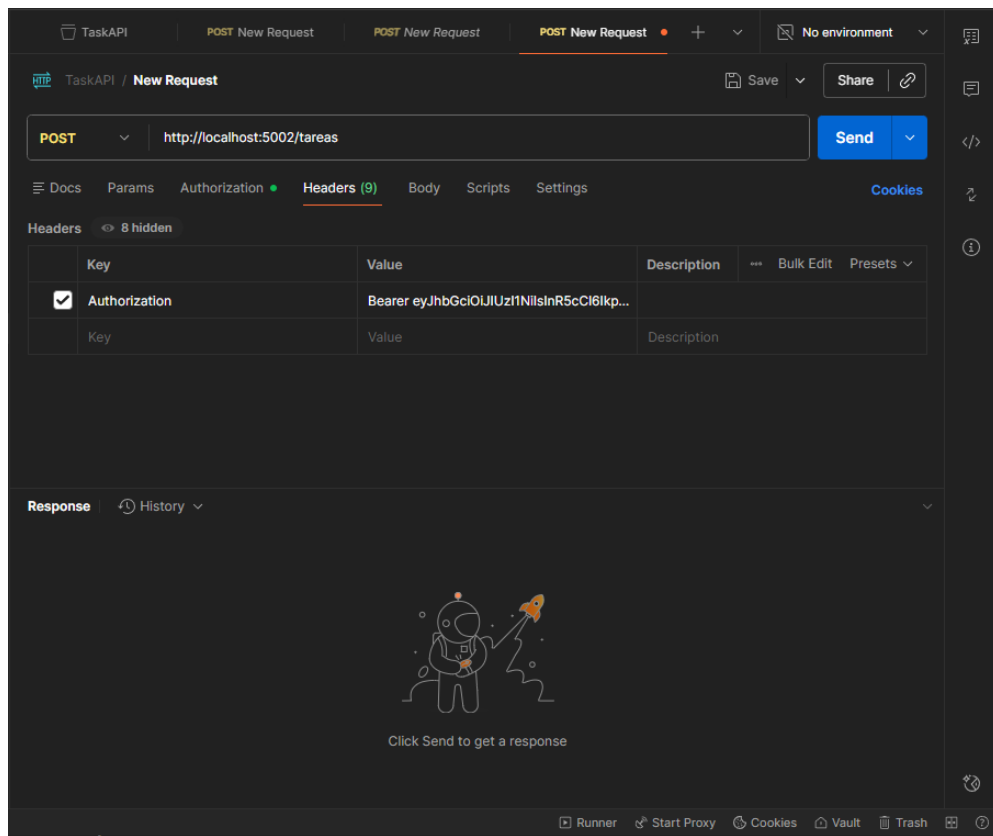


Endpoints postman

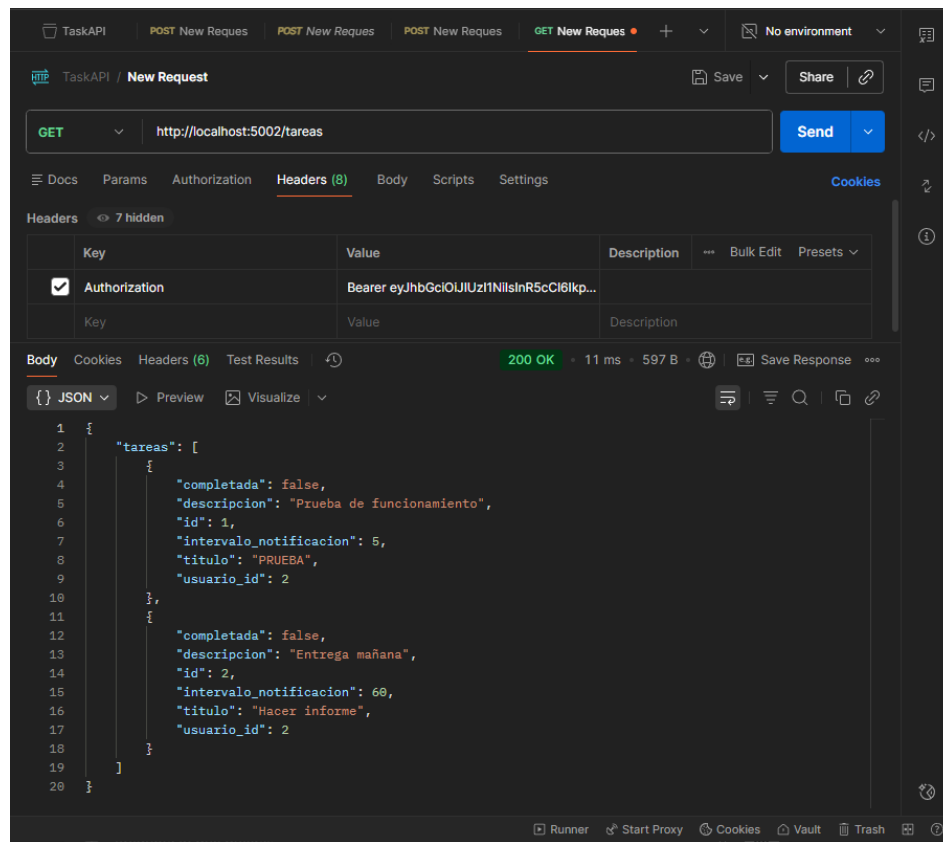
Login:



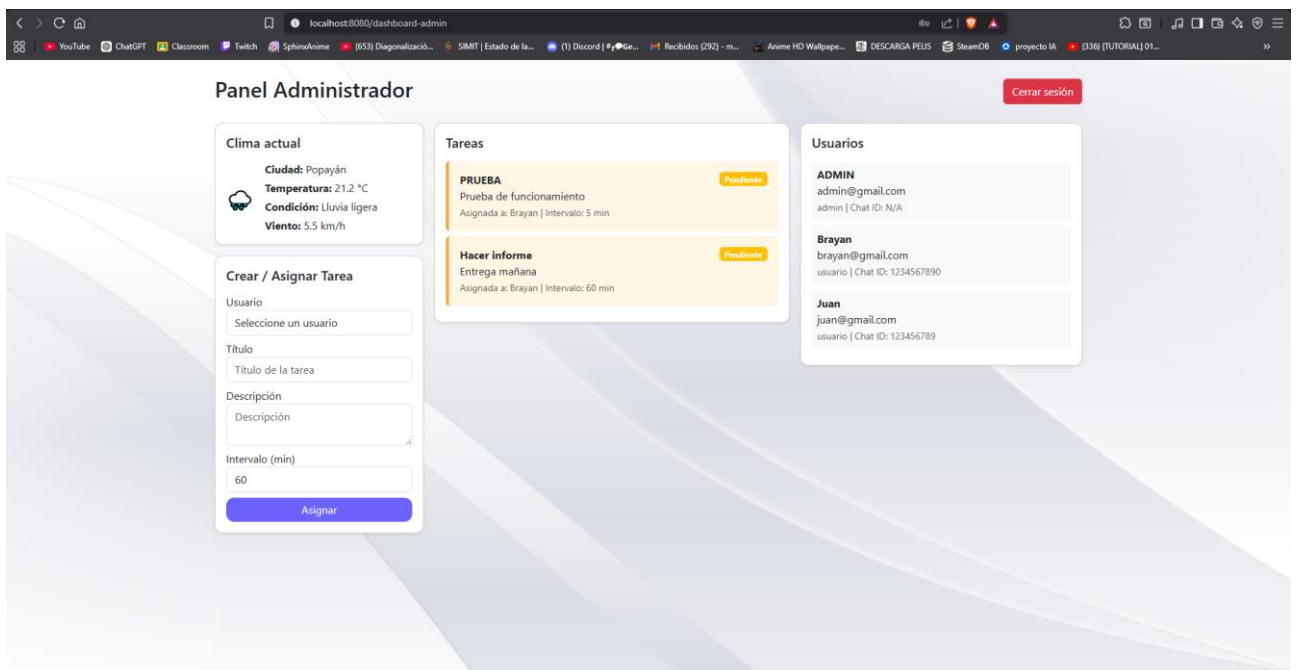
Crear tarea y asignarla a un usuario:



Ver tareas creadas:



Vista en frontend:



Datos SQL en Docker:

Datos almacenados en base de datos usuarios

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
✓ Image taskapi-tareas Built 87.2s
✓ Image taskapi-notificaciones Built 87.2s
✓ Image taskapi-frontend Built 87.2s
✓ Network taskapi_default Created 0.0s
✓ Container taskapi-usuarios-1 Created 0.2s
✓ Container taskapi-tareas-1 Created 0.1s
✓ Container taskapi-notificaciones-1 Created 0.1s
✓ Container taskapi-frontend-1 Created 0.1s
PS C:\PRACTICA\TaskAPI> docker exec -it taskapi-usuarios-1 bash
root@0a8c6d428590:/app# sqlite3 usuarios.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables
usuarios
sqlite> SELECT * FROM usuarios;
1|ADMIN|admin@gmail.com|pbkdf2:sha256:1000000$xx02oXzsF4GjJ8Zj$0d7620d2db50cab9744b09a3be7411b08ef78f3699a884092208823ad6d020f9||admin
2|Brayan|brayan@gmail.com|pbkdf2:sha256:1000000$Lrt837DUtVu1iH8X$a76ff1ab1eec3bf9ea90d1813635f965e7bcc6f92d6f12bcb3c59a906a7bb4d6|1234567890|usuario
3|Juan|juan@gmail.com|pbkdf2:sha256:1000000$cCxLPJFgum1kbFAN$bab4bcc1398736711f249456d7688c4dc532023d6f33470e2246dcd558cbbfc0|123456789|usuario
sqlite>
```

Datos almacenados en base de datos tareas

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\PRACTICA\TaskAPI> docker exec -it taskapi-tareas-1 bash
root@a8017bee2c84:/app# sqlite3 tareas.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables
tareas
sqlite> SELECT * FROM tareas;
1|2|PRUEBA|Prueba de funcionamiento|0|5
2|2|Hacer informe|Entrega mañana|0|60
sqlite>
```